better to use primitive values as they are not nullable. This prevents many bugs. More over primitive values are quite faster than wrapper classes. Also, these wrapper classes are immutable hence they don't have setter method. Every time they need to be modified, a new object will be created. This will create unnecessary object resulting in memory wastage.

Use String wisely: Like wrapper classes, string is also immutable in Java. In any program string needs to be handled with utmost care as misuse of it directly affect the performance of the program. Whenever string object is needed, it is a better idea to instantiate the string directly rather than using an object of string.

Prefer Interfaces over Abstract class: Methods or in simple words functionality can be extended from classes. But Java does not support multiple inheritance by extending multiple class. So, it is better to use interfaces as it is possible to implements multiple interfaces. Again, it is easier to change the implementation of an existing class and add implementation of another interface rather than changing the full hierarchy of the class. But an interface should be used only when there is a clear idea which methods the interface will contain.

Return empty collection instead of null: All the methods return some value unless they are declared as void. The most important thing that needs to be taken care of here is that return type should never be null. The methods should at least return empty value. This will in turn save a lot of checking for null values.

Avoid Null Pointer Exception: Null pointer exceptions are quite common in Java. If a method is called on a null reference of an object this exception is thrown. Whenever it is thrown the program quits, which is unacceptable. For avoiding such scenarios first those object references that may contain null references need to be identified. And then you must put some null check so that they can be eliminated.

Avoid method (T...) signature: In Java var args method can be declared that accepts an Object... arguments. Here T can always be inferred as Objects. These methods cannot be overloaded type safely. So, it is better not to use it.

Proper resource handling: In Java, we can connect easily with external entities like Database and external file systems. But the common mistake most of the developers make is that they forget to close these connections or streams. If a database connection is not closed in a timely fashion, all the connections in the connection pool will be exhausted. Then no other thread will be able to connect to Database. Similarly, after accessing the external file